

tests/dynamic/suites/ memory_soak_suite.sh — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:**

Authored for P10. Honest-SKIPs (topology_unsupported / feature_disabled_by_config, §11.4.69) until the live dynamic stack + RSS hook land; parse-clean + authored today.

Companion (§11.4.18) to the in-source documentation block at the top of the script.

Overview

§11.4.169 **memory soak** suite. It drives a sustained request soak against the live dynamic stack while sampling the control-plane / helper RSS, and asserts the working set does **not grow without bound** (a leak). It records min/max/first/last/mean RSS (§11.4.24 style) and PASSes only when the post-warmup last sample is within MEM_GROWTH_PCT of the post-warmup baseline, citing the sample series.

Prerequisites

- Source library tests/dynamic/lib/analyzer_common.sh + committed tests/lib/evidence.sh.
- curl, awk, POSIX sh, date.
- For a real run: HELIX_DYNAMIC_STACK=1 + HELIX_PROXY_URL, plus HELIX_MEM_RSS_CMD — an operator-supplied command that prints the summed RSS (KB) of the process(es) under test (e.g. the acl-helper + healthd). Config injection §11.4.28 — no hardcoded process discovery in the suite.

Usage examples

```
# Today (no live stack / no RSS hook): honest SKIP, exit 0:  
bash tests/dynamic/suites/memory_soak_suite.sh
```

```
# P10 live run:
```

```
HELIX_DYNAMIC_STACK=1 HELIX_PROXY_URL=http://127.0.0.1:34128 \
MEM_REQUESTS=500 MEM_WARMUP=50 MEM_GROWTH_PCT=20 \
HELIX_MEM_RSS_CMD='ps -o rss= -p "$(pgrep -f external_acl)'"' \
bash tests/dynamic/suites/memory_soak_suite.sh
```

Env: MEM_REQUESTS (default 500), MEM_WARMUP (default 50), MEM_TARGET (default http://target-a.internal/), MEM_GROWTH_PCT (max allowed last-vs-baseline growth %, default 20).

Edge cases

- **Live stack absent** → topology_unsupported SKIP, exit 0.
- **RSS hook not configured** (HELIX_MEM_RSS_CMD empty) → feature_disabled_by_config SKIP, exit 0.
- **Growth above MEM_GROWTH_PCT** (or no post-warmup samples) → FAIL citing memory.evidence.

§11.4.115 RED_MODE polarity

RED_MODE	What it asserts	PASS means
0 (default)	GREEN guard	working set bounded (growth $\leq$ MEM_GROWTH_PCT)
1	RED baseline	the pre-fix stack shows unbounded growth (growth > threshold) → leak reproduced

A RED_MODE=1 run where the working set stayed bounded is a finding (no leak to reproduce), not a pass.

Internal behaviour

- #!/usr/bin/env bash, POSIX-clean (sh -n + bash -n, §11.4.67).
- Soaks MEM_REQUESTS proxied requests, sampling RSS every 10 requests into rss.series; awk computes baseline (first post-warmup sample), last, min/max/mean, and growth %.
- GREEN requires baseline > 0 AND growth $\leq$ MEM_GROWTH_PCT. Evidence (memory.evidence + the series) under qa-results/p9-harness/memory_soak_<run-id>/.
- Resources: shell+curl only; bounded request count; §12.6 60% host ceiling.

Related

- tests/dynamic/lib/analyzer_common.sh — sourced base.

- tests/dynamic/suites/run_all_suites.sh — orchestrates this suite.
- Constitution §11.4.169 / §11.4.24 / §11.4.85 / §11.4.69 / §11.4.115 / §12.6; design spec §13.

Last verified

2026-07-01 — run with no live stack: honest SKIP, exit 0; sh -n + bash -n parse-clean. Live soak + RSS sampling is exercised in **P10**.